МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»
Навчально-науковий Фізико-технічний інститут

ЗВІТ

з комп’ютерного практикуму №4

з дисципліни «Криптографія»

Виконали:

студентки третього курсу

групи ФБ-41
Борис Марія,

Касян Софія

Перевірила:

Селюх П. В.

м. Київ – 2026

Тема. Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Мета: ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

Порядок виконання роботи

1. Написати функцію пошуку випадкового простого числа з заданого інтервалу або заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на простоту. В якості датчика випадкових чисел використовуйте вбудований генератор псевдовипадкових чисел вашої мови програмування. В якості тесту перевірки на простоту рекомендовано використовувати тест Міллера-Рабіна із попередніми пробними діленнями. Тести необхідно реалізовувати власноруч, використання готових реалізацій тестів не дозволяється.

Відповідно до завдання я реалізовала функцію генерації випадкового простого числа заданої бітової довжини з використанням вбудованого генератора псевдовипадкових чисел Python, попередніх пробних ділень та імовірнісного тесту Міллера–Рабіна. Для подальшої побудови криптосистеми RSA було обрано довжину простого числа 256 біт.

Спочатку я реалізовала функцію trial_division(n), яка виконує перевірку кандидата на подільність на малі прості числа 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37. Якщо число ділиться на одне із цих чисел і не дорівнює йому, воно відхиляється як складене. Попередні пробні ділення дозволяють відкинути частину складених кандидатів без виконання складніших обчислень.

```python
def trial_division(n):
    primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
    for p in primes:
        if n == p:
            return True
        if n % p == 0:
            return False
    return True
```

Для ефективного обчислення степенів за модулем було реалізовано функцію modular_power(x, a, m) на основі схеми Горнера, наведеної в методичці. Показник степеня a подається у двійковому вигляді, після чого результат послідовно обчислюється за допомогою піднесення до квадрата та множення за модулем

```python
def modular_power(x, a, m):
    y = 1
    for bit in bin(a)[2:]:
        y = (y * y) % m
        if bit == "1":
            y = (y * x) % m
    return y
```

2. Схема Горнера швидкого піднесення до степеня

Потрібно обчислити вираз $x^{\alpha} (\mathrm{mod}\, m), 0 \le \alpha,\ x < m,\ \alpha, x$ – цілі невід'ємні числа. Представимо α у двійковій формі: $\alpha = \sum_{i=0}^{k-1} \alpha_i 2^i$, $\alpha_i \in \{0, 1\}$. Тоді піднесення до степеня α можна звести до не більш ніж $k-1$ піднесення в квадрат за модулем і не більш ніж $k-1$ множень за модулем за допомогою однієї з двох формул:

$$x^{\alpha} (\mathrm{mod}\, m) = (...(...(x^{\alpha_{k-1}})^2 x^{\alpha_{k-2}})^2 ... x^{\alpha_1})^2 x^{\alpha_0} (\mathrm{mod}\, m), \text{ або}$$
$$x^{\alpha} (\mathrm{mod}\, m) = x^{\alpha_0} (x^2)^{\alpha_1} (x^{2^2})^{\alpha_2} ... (x^{2^{k-1}})^{\alpha_{k-1}} (\mathrm{mod}\, m)$$

Далі було реалізовано функцію miller_rabin(n, k=40), яка перевіряє число на ймовірну простоту. Відповідно до алгоритму Міллера–Рабіна число n - 1 розкладається у вигляді: n - 1 = d * 2^s, де d - не парне число.

Після цього випадково вибирається основа x з інтервалу 1 < x < n. За допомогою алгоритму Евкліда перевіряється умова gcd(x, n) = 1. Якщо найбільший спільний дільник більший за 1, число визнається складеним.

```python
def miller_rabin(n, k=40, details=False, first_base=None):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    if not trial_division(n):
        return False
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    if details:
        print("n =", n)
        print("n - 1 = d * 2^s")
        print("d =", d)
        print("s =", s)
    for i in range(k):
        if i == 0 and first_base is not None:
            x = first_base
        else:
            x = random.randrange(2, n - 1)
        if gcd(x, n) != 1:
            return False
        y = modular_power(x, d, n)
        if details and i == 0:
            print("\nОснова x =", x)
            print("gcd(x, n) =", gcd(x, n))
            print("x^d mod n =", y)
        if y == 1 or y == n - 1:
            if details and i == 0:
                print("Перевірку за цією основою пройдено")
            continue
        passed = False
        for r in range(1, s):
            y = (y * y) % n
            if details and i == 0:
```

```python
                print("Перевірку за цією основою пройдено")
            continue
        passed = False
        for r in range(1, s):
            y = (y * y) % n
            if details and i == 0:
                print("r =", r)
                print("x^(d*2^r) mod n =", y)
            if y == n - 1:
                passed = True
                break
            if y == 1:
                return False
        if not passed:
            return False
    return True
```

Для кожної вибраної основи обчислюється значення x^d mod n. Якщо результат дорівнює 1 або n - 1, число проходить перевірку за цією основою. В іншому випадку виконуються послідовні піднесення до квадрата за модулем, доки не буде отримано n - 1 або не буде встановлено, що число складене.

3. Псевдопрості числа

Для побудови, формального опису та оцінки коректності та ефективності тестів на простоту використовуються так звані *псевдопрості числа* – складені числа, що зберігають ті чи інші властивості простих.

Непарне число p називається *псевдопростим за основою* $a \in N$, якщо $\gcd(a, p) = 1$ [1] і $a^{p-1} \equiv 1 \pmod{p}$.

Непарне число p називається *псевдопростим числом Ойлера*[2] *за основою* $a \in N$, якщо $\gcd(a, p) = 1$ та $\left(\frac{a}{n}\right) \equiv a^{\frac{n-1}{2}} \pmod{n}$, де $\left(\frac{a}{n}\right)$ – символ Якобі.

Нехай p – непарне число, $p - 1 = d \cdot 2^s$, d – непарне. Число p називається *сильним псевдопростим числом за основою* $a \in N$, якщо $a^d \equiv 1 \pmod{p}$ або $a^{d \cdot 2^r} \equiv -1 \pmod{p}$ при якомусь $r, 0 \le r < s$.

Для перевірки було встановлено k = 40 раундів. Відповідно до методички верхня оцінка ймовірності помилкового прийняття складеного числа становить 4^(-k), тобто для 40 раундів - 4^(-40).

На основі реалізованих перевірок було створено функцію generate_prime(bits=256). Спочатку за допомогою random.getrandbits() генерується випадковий кандидат. Старший біт встановлюється рівним 1 для забезпечення заданої бітової довжини, а молодший - рівним 1, щоб число було непарним. Після цього кандидат проходить перевірку на простоту. Якщо перевірка не пройдена, число зберігається у списку відхилених кандидатів, а генерація повторюється

```
def generate_prime(bits=256):
    if bits < 2:
        raise ValueError("Мінімальна довжина — 2 біти")
    rejected = []
    while True:
        n = random.getrandbits(bits)
        n |= (1 << (bits - 1))
        n |= 1
        if miller_rabin(n):
            return n, rejected
        rejected.append(n)
```

У результаті виконання програми отримано 256-бітне ймовірно просте число:

p =
1147482440922293807623668155687892248627950437894387024713235803
67482101979953

Під час генерації було відхилено 62 кандидати. Наприклад, перший відхилений кандидат:
6454214314129669319429396889840726436928860796338576404342925748
9177102727429 є складеним, оскільки ділиться на 3

```
ЗАВДАННЯ 1. ГЕНЕРАЦІЯ ПРОСТОГО ЧИСЛА

Згенероване просте число:
p = 114748244092229380762366815568789224862795043789438702471323580367482101979953

Бітова довжина:
256 біт

Кількість відхилених кандидатів:
62

Перші 5 відхилених кандидатів:
1 – 6454214314129669319429396889840726436928860796338576404342925748917710272742 9
2 – 1119339901471503659314806891904394427300638015881023447369269262587578409921 51
3 – 6470520713402119285933308949951532080365993215426992120913321466680741609818 3
4 – 1035785129329166903121653266998529487080111039485468715661766961739658727966 93
5 – 1139492495626094064707879310989823766773250692730272324396523439476682066937 93
```

Доповнення до п. 1. Для одного зі згенерованих простих чисел навести проміжні результати перевірки тестом Міллера-Рабіна для однієї з використаних основ.

Для отриманого числа було додатково виведено проміжні результати перевірки тестом Міллера–Рабіна за однією з випадково вибраних основ.

Було отримано такі параметри:

d =
7171765255764336297647925973049326553924690236839918904457723772
967631373747

s = 4

Таким чином, виконується співвідношення p - 1 = d * 2^4, причому d є непарним числом.

Для перевірки було вибрано основу:

x =
675861334658459520948993446624043817188353086855231096806996195395920658064 39

У результаті обчислення найбільшого спільного дільника отримано gcd(x, p) = 1, тому можна виконувати подальшу перевірку.

Піднесення основи x до степеня d за модулем p дало результат:

x^d mod p = 1.

Отже, виконується перша умова сильної псевдопростоти, тому число пройшло перевірку за вибраною основою без необхідності подальших піднесень до квадрата

```
Перевірка тестом Міллера–Рабіна:
n = 114748244092229380762366815568789224862795043789438702471323580367482101979953
n − 1 = d * 2^s
d = 7171765255764336297647925973049326553924690236839918904457723772967631373747
s = 4

Основа x = 675861334658459520948993446624043817188353086855231096806996195395920658064 39
gcd(x, n) = 1
x^d mod n = 1
Перевірку за цією основою пройдено

Підсумковий результат:
Число пройшло тест: True
```

Після виконання 40 раундів тесту програма повернула результат True. Таким чином, згенероване число пройшло перевірку Міллера–Рабіна та було прийняте як ймовірно просте для подальшого використання під час генерації ключів RSA

2. За допомогою цієї функції згенерувати дві пари простих чисел p, q і p_1, q_1 довжини щонайменше 256 біт. При цьому пари чисел беруться так, щоб $pq \leq p_1q_1$; p і q - прості числа для побудови ключів абонента A, p_1 і q_1 - абонента B.

Відповідно до завдання за допомогою функції генерації простих чисел, реалізованої в попередньому пункті, було сформовано дві пари простих чисел p, q та p1, q1 довжиною щонайменше 256 біт. Перша пара призначена для побудови ключів абонента A, а друга - для B.

Для абонента A було використано раніше згенероване число p та додатково згенеровано число q. Для абонента B було згенеровано числа p1 та q1. Кожен кандидат проходив попередню перевірку методом пробних ділень і 40 раундів імовірнісного тесту Міллера–Рабіна.

Для забезпечення заданої бітової довжини під час генерації встановлювався старший біт числа, а для отримання непарного кандидата - молодший. Якщо кандидат не проходив перевірку на простоту, він відхилявся, після чого генерація повторювалася.

Відповідно до методички пари простих чисел необхідно вибрати так, щоб виконувалася умова pq <= p1q1. Для цього було обчислено добутки n = pq для абонента A та n1 = p1q1 для абонента B

```
p = 114748244092229380762366815568789224862795043789438702471323580367482101979953
q, rejected_q = generate_prime(256)
p1, rejected_p1 = generate_prime(256)
q1, rejected_q1 = generate_prime(256)
n = p * q
n1 = p1 * q1

while n1 < n:
    p1, rejected_new_p1 = generate_prime(256)
    q1, rejected_new_q1 = generate_prime(256)
    rejected_p1.extend(rejected_new_p1)
    rejected_q1.extend(rejected_new_q1)
    n1 = p1 * q1
```

У програмі було передбачено перевірку цієї нерівності. Якщо n1 < n, повторно генерувалася пара чисел для абонента B, доки необхідна умова не виконувалася.

У результаті генерації для абонента A отримано:

```
Абонент A:
p = 114748244092229380762366815568789224862795043789438702471323580367482101979953
q = 71605663814313799781788654033119510661804200390579721225978662695715637312979
```

Для абонента B отримано:

```
Абонент B:
p1 = 10315258775163154097298046633823752021555954347012464065426051744253133469084 1
q1 = 10665639789079205719677838647014152619019384831300214071437782364490755721789 1
```

Бітова довжина кожного із чотирьох чисел становить 256 біт

```
Бітові довжини:
p: 256
q: 256
p1: 256
q1: 256
```

Для абонента A було обчислено добуток:



Для абонента B було отримано:



Обидва добутки мають довжину 512 біт. Порівняння отриманих значень показало, що n < n1, тобто умова pq <= p1q1 виконується

```
Перевірка умови p*q <= p1*q1:
True
```

Додатково було виконано повторну перевірку простоти всіх чотирьох чисел за допомогою реалізованого тесту Міллера–Рабіна. Для кожного числа програма повернула результат True, що підтверджує успішне проходження імовірнісної перевірки на простоту.

Під час генерації числа q для абонента A було відхилено 102 кандидати. Для параметрів абонента B накопичено 446 відхилені кандидати під час генерації p1 та 210 - під час генерації q1. Ці кількості включають повторні спроби формування пари абонента B для виконання умови n <= n1.

```
Перевірка простоти:
p: True
q: True
p1: True
q1: True

Кількість відхилених кандидатів:
q: 102
p1: 446
q1: 210

Результати збережено у файли:
rsa_primes.txt
rejected_candidates.txt
```

Для подальшого використання параметри p, q, p1, q1, n та n1 було збережено у файлі rsa_primes.txt. Перелік відхилених кандидатів поточної генерації записано у файл rejected_candidates.txt.

(Під час виконання програми виникла проблема зі зчитуванням раніше збережених параметрів RSA. При запуску програми з іншого робочого каталогу Python не знаходив файл rsa_primes.txt, унаслідок чого виникала помилка FileNotFoundError. Для її усунення було налаштовано роботу з файлами відносно каталогу, в якому розташована програма. Після цього зчитування збережених простих чисел, ключів RSA та результатів попередніх завдань виконувалося коректно.

Також для забезпечення відтворюваності експериментальних результатів було передбачено можливість використання раніше збережених параметрів RSA. Це дозволило повторно виконувати перевірки шифрування, цифрового підпису та протоколу передавання ключів без зміни значень)

3. Написати функцію генерації ключових пар для RSA. Після генерування функція повинна повертати та/або зберігати секретний ключ (d, p, q) та відкритий ключ (n, e). За допомогою цієї функції побудувати схеми RSA для абонентів A і B - тобто, створити та зберегти для подальшого використання відкриті ключі (e, n), (e_1, n_1) та секретні d і d_1.

Відповідно до завдання було реалізовано функцію GenerateKeyPair(), призначену для генерації відкритого та секретного ключів криптосистеми RSA. Для побудови ключових пар використано прості числа p, q, p1 та q1, отримані в попередньому завданні.

```
248  def GenerateKeyPair(p, q):
249      if p == q:
250          raise ValueError("Числа p і q повинні бути різними")
251      n = p * q
252      phi = (p - 1) * (q - 1)
253      e = 65537
254      if e >= phi or gcd(e, phi) != 1:
255          while True:
256              e = random.randrange(2, phi)
257              if gcd(e, phi) == 1:
258                  break
259      d = modular_inverse(e, phi)
260      public_key = (n, e)
261      private_key = (d, p, q)
262      return public_key, private_key
```

Спочатку було реалізовано функцію extended_euclid(a, b), яка виконує розширений алгоритм Евкліда для знаходження найбільшого спільного дільника двох чисел та коефіцієнта, необхідного для обчислення оберненого елемента за модулем.

```
def extended_euclid(a, b):
    old_r, r = a, b
    old_s, s = 1, 0
    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
    return old_r, old_s
```

На основі розширеного алгоритму Евкліда реалізовано функцію modular_inverse(a, m). Вона знаходить число x, для якого виконується співвідношення $a*x \equiv 1 \pmod m$. Якщо найбільший спільний дільник a та m не дорівнює 1, обернений елемент не існує і програма повідомляє про помилку

```
def modular_inverse(a, m):
    g, x = extended_euclid(a, m)
    if g != 1:
        raise ValueError("Обернений елемент не існує")
    return x % m
```

Для побудови RSA у функції GenerateKeyPair(p, q) спочатку обчислюється модуль n як добуток двох простих чисел:

$n = p*q$.

Після цього обчислюється функцію Ойлера $\varphi(n) = (p-1)(q-1)$.

Далі вибирається відкрита експонента e, яка повинна задовольняти умови 2 <= e <= φ(n)-1 та gcd(e, φ(n)) = 1. Відповідно до зауваження у методичці для генерації ключів було використано значення e = 65537.

У програмі додатково перевіряється взаємна простота e та φ(n). Якщо вибране значення не задовольняє необхідні умови, передбачено вибір іншої допустимої експоненти.

Секретна експонента d обчислюється як обернений елемент до e за модулем φ(n) за допомогою раніше реалізованої функції modular_inverse(). Отже, для коректної роботи RSA повинно виконуватися співвідношення:

e*d ≡ 1 (mod φ(n)).

У результаті функція GenerateKeyPair() повертає відкритий ключ (n, e) та секретний ключ (d, p, q). Аналогічні обчислення виконуються для абонента B з використанням чисел p1 та q1.

Для абонента A було отримано такі параметри:

```
Абонент A:
n = 8216624189750996622600210007550177510009771095868593182601656766415108425928771780615025679433968248525388233074214899381758656746217
28780214342064470998 7
phi(n) = 8216624189750996622600210007550177510009771095868593182601656766415108425928585426707119136253424093055786324338690300137578638 3
22519985559080222905417056
e = 65537
d = 2836206607268368182804552402319302312151014567812675511189323273415667223268633894181431086261576816654836172378806658371794619163709
768730907926859123009
```

Для абонента B було отримано:

```
Абонент B:
n1 = 1100188344270285688018303615840298680204001769914746281425639601921289755778947390587411716359798298529645785970236802299749390768362097663321463255903633 1
phi(n1) = 1100188344270285688018303615840298680204001769914746281425639601921289755778926409688847473999981322644364948065596226960571078090225233829212719366712760 0
e1 = 65537
d1 = 4666697557466724421505534616589783330178532005105517780391436180144030688145379749292807258453331351575304044910134878210429458166076739478741541200775867 3
```

Для однієї зі згенерованих пар ключів навести значення основних параметрів та перевірити виконання співвідношень, необхідних для коректної роботи RSA.

Для перевірки коректності згенерованих ключів було виконано перевірку математичних співвідношень RSA.

Для абонента A підтверджено правильність обчислення n = pq та φ(n) = (p-1)(q-1). Також перевірено допустимість відкритої експоненти e та взаємну простоту e і φ(n).

У результаті отримано:

```
Перевірка параметрів RSA для абонента A:
n = p*q: True
phi(n) = (p-1)*(q-1): True
2 <= e <= phi(n)-1: True
gcd(e, phi(n)) = 1
e*d mod phi(n) = 1
```

Це підтверджує, що секретна експонента d є оберненим елементом до e за модулем φ(n).

Аналогічну перевірку було виконано для абонента B:

```
Перевірка параметрів RSA для абонента B:
n1 = p1*q1: True
2 <= e1 <= phi(n1)-1: True
gcd(e1, phi(n1)) = 1
e1*d1 mod phi(n1) = 1
```

Таким чином, для обох абонентів виконуються необхідні співвідношення RSA. Додатково перевірено умову n <= n1, необхідну для подальшої реалізації протоколу конфіденційного передавання ключа. Програма повернула True.

Для подальшого використання згенеровані ключові пари було збережено у файлі rsa_keys.json. Для кожного абонента файл містить відкритий ключ (n, e) та секретні параметри (d, p, q)

```
Перевірка умови n <= n1:
True

Ключові пари збережено у файл rsa_keys.json
```

4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів А і В. Кожна з операцій (шифрування, розшифрування, створення цифрового підпису, перевірка цифрового підпису) повинна бути реалізована окремою процедурою, на вхід до якої повинні подаватись лише ті ключові дані, які необхідні для її виконання.

За допомогою датчика випадкових чисел вибрати відкрите повідомлення М і знайти криптограму для абонентів А і В, перевірити правильність розшифрування. Скласти для А і В повідомлення з цифровим підписом і перевірити його.

Відповідно до завдання було реалізовано програму шифрування, розшифрування, створення та перевірки цифрового підпису за алгоритмом RSA. Для виконання операцій використано ключові пари абонентів А і В, згенеровані на попередньому етапі.

Згідно з вимогами методички кожну операцію було реалізовано у вигляді окремої процедури, яка отримує лише необхідні ключові дані. Для цього я створила чотири функції: Encrypt(), Decrypt(), Sign() та Verify().

Спочатку було реалізовано функцію Encrypt(message, public_key), яка виконує шифрування повідомлення за допомогою відкритого ключа адресата (n, e). На вхід функції передається повідомлення M та відкритий ключ, після чого перевіряється допустимість значення M: 0 <= M < n.

Шифрування здійснюється відповідно до формули з методички:

C = M^e mod n, де M - відкрите повідомлення, e - відкрита експонента, n - модуль RSA, C - отримана криптограма.

Для обчислення степеня за модулем використано вбудовану функцію Python pow(message, e, n), яка дозволяє ефективно виконувати модульне піднесення до степеня.

Далі було реалізовано функцію Decrypt(ciphertext, private_key), призначену для розшифрування криптограми за допомогою секретного ключа абонента. Функція отримує криптограму C та секретні параметри (d, p, q), на основі яких обчислюється модуль n = p*q.

Розшифрування виконується за формулою:

Розшифрування криптограми $0 \le C \le n-1$ здійснюється за допомогою особистого ключа d за формулою:

$$M = C^d \bmod n \ .$$

Після розшифрування отримане повідомлення порівнюється з початковим. Якщо значення збігаються, операція шифрування та розшифрування виконана правильно.

Для створення цифрового підпису було реалізовано функцію Sign(message, private_key). Вона отримує повідомлення M та секретний ключ абонента, після чого обчислює підпис:

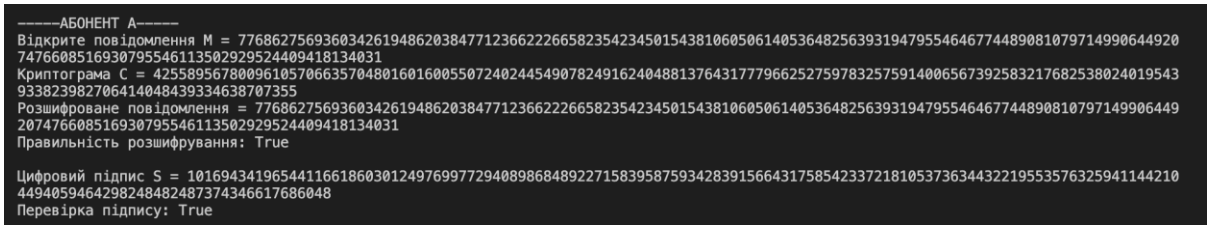
S = M^d mod n, де S - цифровий підпис повідомлення.

Для перевірки цифрового підпису реалізовано функцію Verify(message, signature, public_key), яка використовує повідомлення, отриманий підпис S та відкритий ключ підписувача (n, e).

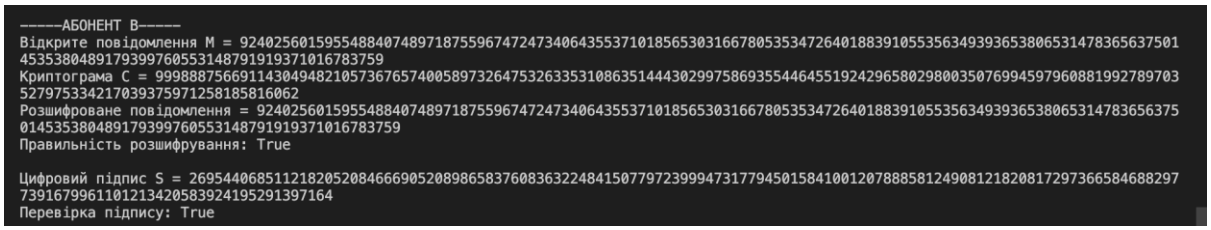
Перевірка виконується шляхом обчислення S^e mod n та порівняння отриманого значення з початковим повідомленням M. Якщо значення збігаються, функція повертає True, інакше - False.

Для практичної перевірки реалізованих процедур за допомогою генератора випадкових чисел було вибрано окремі відкриті повідомлення для абонентів A і B.

Для абонента A було отримано відкрите повідомлення M, а після шифрування за відкритим ключем абонента A отримано криптограму C. Під час розшифрування за секретним ключем отримано повідомлення, яке повністю збігається з початковим. Перевірка правильності розшифрування повернула True. Для цього самого повідомлення було сформовано цифровий підпис S. Перевірка цифрового підпису за допомогою відкритого ключа абонента A також повернула True



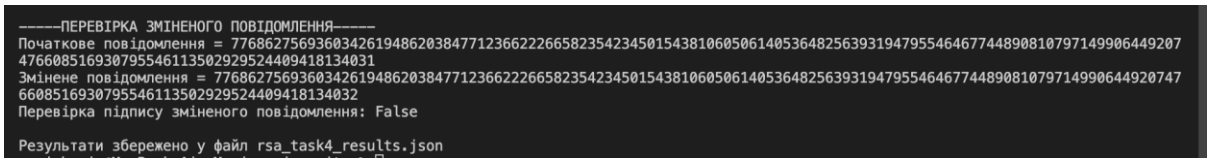
Аналогічні операції було виконано для абонента B



Для додаткової перевірки роботи цифрового підпису було змінено початкове повідомлення абонента A, збільшивши його значення на 1.

Після зміни повідомлення перевірку виконано з використанням попереднього цифрового підпису. Функція Verify() повернула False, оскільки підпис було створено для іншого значення повідомлення.

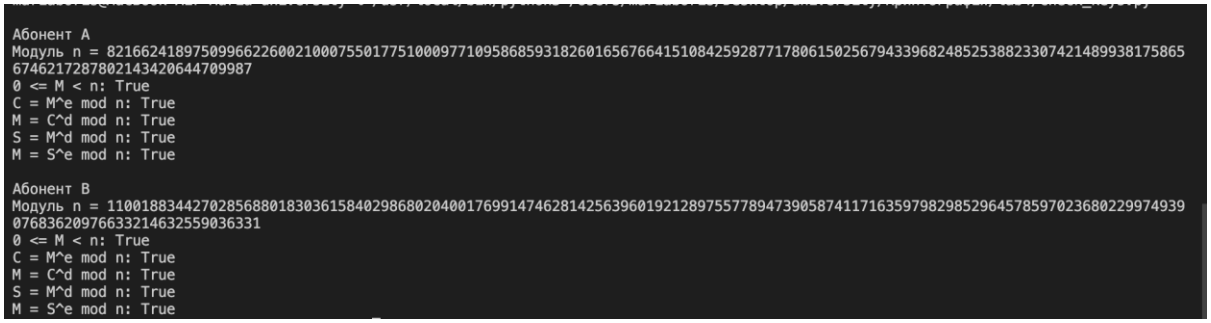
Це підтверджує, що зміна підписаного повідомлення виявляється під час перевірки його цифрового підпису



Результати шифрування, розшифрування та цифрового підписування було збережено у файлі rsa_task4_results.json для подальшого аналізу.

Для додаткового підтвердження правильності обчислень було перевірено результати за допомогою окремої програми check_keys.py, яка зчитує збережені ключі RSA та результати виконання операцій.

Для обох абонентів було підтверджено виконання співвідношень C = M^e mod n, M = C^d mod n, S = M^d mod n та M = S^e mod n. Усі перевірки повернули True.



5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA. Протоколи роботи кожного учасника (відправника та приймаючого) повинні бути реалізовані у вигляді окремих процедур, на вхід до яких повинні подаватись лише ті ключові дані, які необхідні для виконання. Перевірити роботу програм для випадково обраного ключа 0 < k < n.



Відповідно до завдання було реалізовано протокол конфіденційного розсилання ключів із підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA. Програму оформлено окремим файлом rsa_lab5.py, який містить сім високорівневих процедур: GenerateKeyPair(), Encrypt(), Decrypt(), Sign(), Verify(), SendKey() та ReceiveKey(). Кожна процедура приймає лише ті дані, які необхідні для її виконання. Ключові пари абонентів A і B зчитуються з файлу rsa_keys.json, створеного на третьому етапі роботи.

Протоколи відправника і отримувача реалізовано у вигляді окремих процедур SendKey(k, sender_private_key, receiver_public_key) та

ReceiveKey(k1, S1, receiver_private_key, sender_public_key). Функція SendKey() отримує ключ k, секретний ключ відправника та відкритий ключ отримувача. Функція ReceiveKey() отримує пару (k1, S1), секретний ключ отримувача та відкритий ключ відправника. Якщо підпис не проходить перевірку, функція повертає None, тобто ключ відхиляється.

Відправником виступає абонент A, отримувачем – абонент B. Перед початком роботи програма перевіряє умову n <= n1, яка гарантує, що підпис S < n коректно зашифрується за модулем n1. Перевірка повернула True.

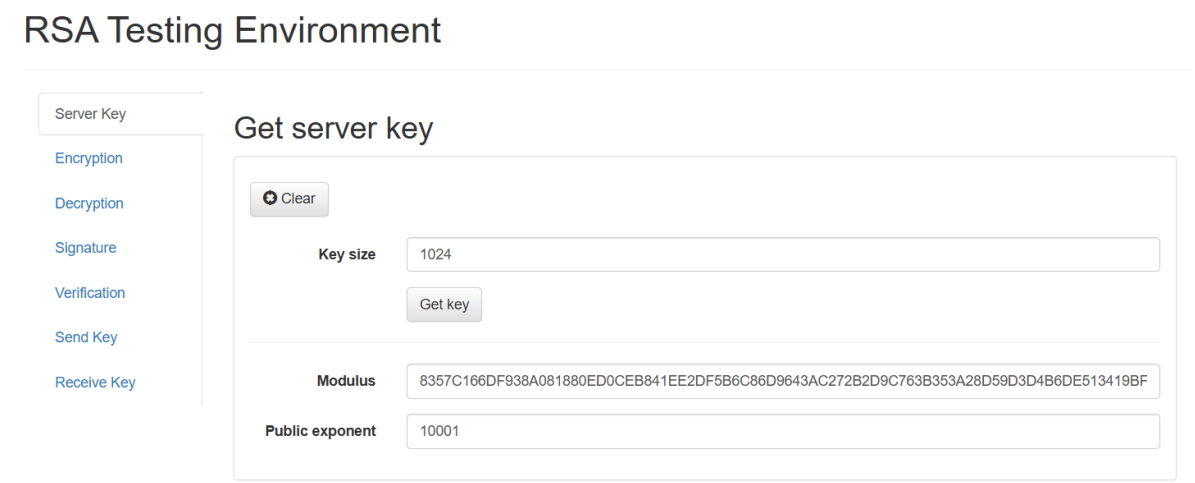
Абонент A обирає випадковий ключ k з інтервалу 0 < k < n та обчислює підпис S = k^d mod n за допомогою свого секретного ключа. Потім за відкритим ключем абонента B він шифрує ключ та підпис: k1 = k^e1 mod n1 і S1 = S^e1 mod n1. Пара (k1, S1) передається абоненту B по відкритому каналу.

Абонент B розшифровує отримані значення за допомогою свого секретного ключа d1: k = k1^d1 mod n1 та S = S1^d1 mod n1. Після цього він перевіряє підпис за відкритим ключем абонента A, тобто виконання рівності k = S^e mod n. Значення k та S, отримані абонентом B, повністю збіглися зі значеннями, які сформував абонент A. Перевірка підпису повернула True, а отриманий ключ збігається з надісланим.

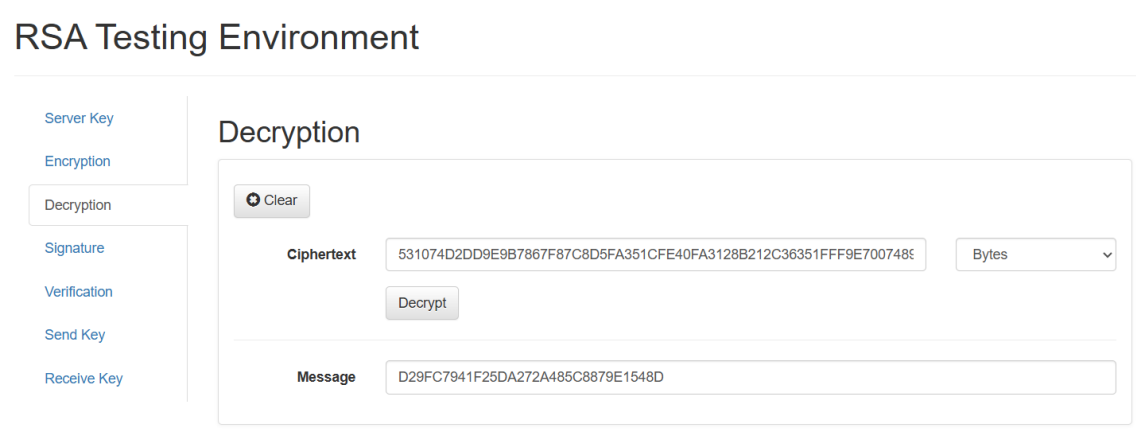
Конфіденційність передачі забезпечується шифруванням за відкритим ключем абонента B (e1, n1): розшифрувати k1 та S1 може лише власник секретного ключа d1, тобто абонент B. Підтвердження справжності відправника забезпечується цифровим підписом S = k^d mod n: створити його можна лише за секретним ключем абонента A (d), а перевірити за його відкритим ключем (e, n).

Для додаткової перевірки було змінено S1 на одиницю. У цьому випадку функція ReceiveKey() повернула None, тобто ключ було відхилено. Так само ключ відхиляється, якщо перевіряти підпис за чужим відкритим ключем (ключем абонента B замість A). Це підтверджує, що спотворення переданих даних виявляється під час перевірки підпису.

Кожну операцію рекомендується перевіряти шляхом взаємодії із тестовим середовищем, розташованим за адресою: http://asymcryptwebservice.appspot.com/?section=rsa.







Спочатку на вкладці Server Key було згенеровано ключ сервера розміром 1024 біти. Сервер повернув модуль n_s та відкриту експоненту e_s = 10001 (hex).

Далі програмою було обрано випадкове відкрите повідомлення M = D29FC7941F25DA272A485C8879E1548D (hex) та зашифровано його власною функцією Encrypt() за відкритим ключем сервера (n_s, e_s). Отриманий шифртекст C було подано на вхід вкладки Decryption тестового середовища (режим Bytes).

Сервер розшифрував шифртекст своїм секретним ключем і повернув повідомлення D29FC7941F25DA272A485C8879E1548D, яке повністю збігається з вихідним повідомленням M. Отже, власна реалізація шифрування сумісна з реалізацією сервера.

```
=== Перевірка із тестовим середовищем ===
1 - Encrypt (наша) -> Decrypt на сервері
2 - Encrypt на сервері -> Decrypt (наша)
3 - Sign на сервері -> Verify (наша)
4 - Sign (наша) -> Verify на сервері
5 - SendKey (наша) -> ReceiveKey на сервері
6 - SendKey на сервері -> ReceiveKey (наша)
0 - вихід
Оберіть пункт: 2

[2] Encrypt на сервері -> Decrypt (наша реалізація)

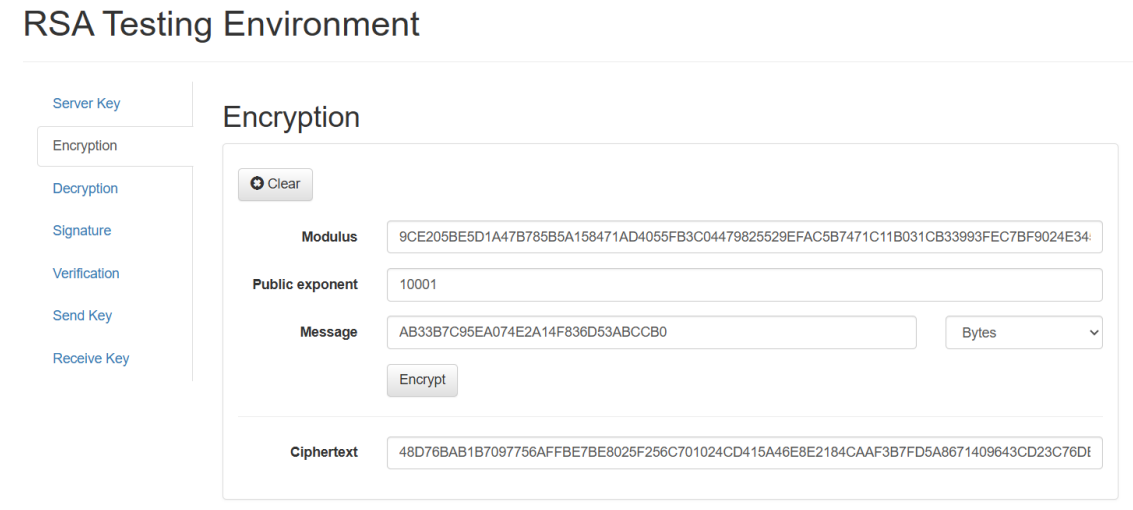
Вкладка Encryption (режим Bytes): введіть Modulus, Public exponent, Message
Modulus (n абонента A):
9CE205BE5D1A47B785B5A158471AD4055FB3C04479825529EFAC5B7471C11B031CB33993FEC7BF9024E3459382B7D16C827105B4AD718F7590FB0F88772DD663

Public exponent (e):
10001

Message M:
AB33B7C95EA074E2A14F836D53ABCCB0

(значення також записано у файл server_values.txt)
Натисніть Encrypt, скопіюйте Ciphertext із сайту.
  Ciphertext із сайту, hex: 48D76BAB1B7097756AFFBE7BE8025F256C701024CD415A46E8E2184CAAF3B7FD5A8671409643CD23C76DB0D70E6F16F75A59409AA1774342B92BF5580FF725B1

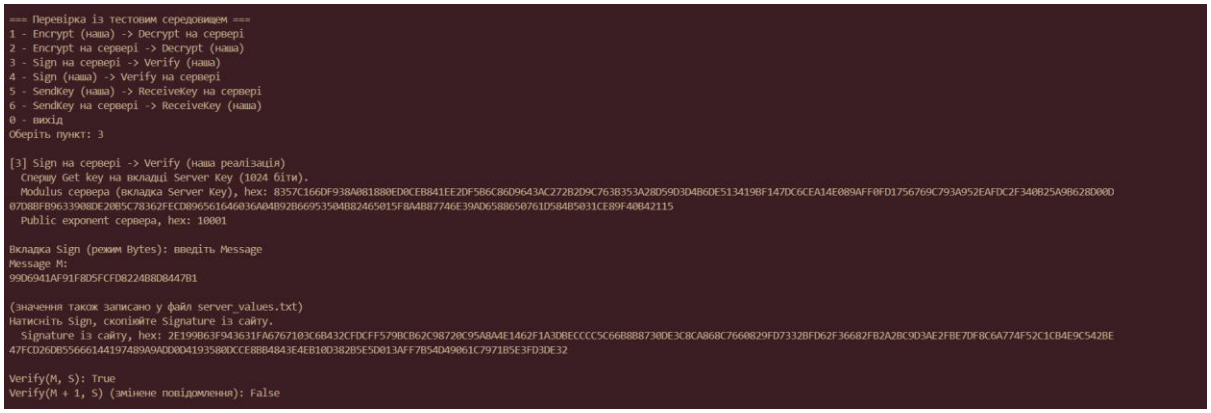
Розшифровано: AB33B7C95EA074E2A14F836D53ABCCB0
Збігається з M: True
```

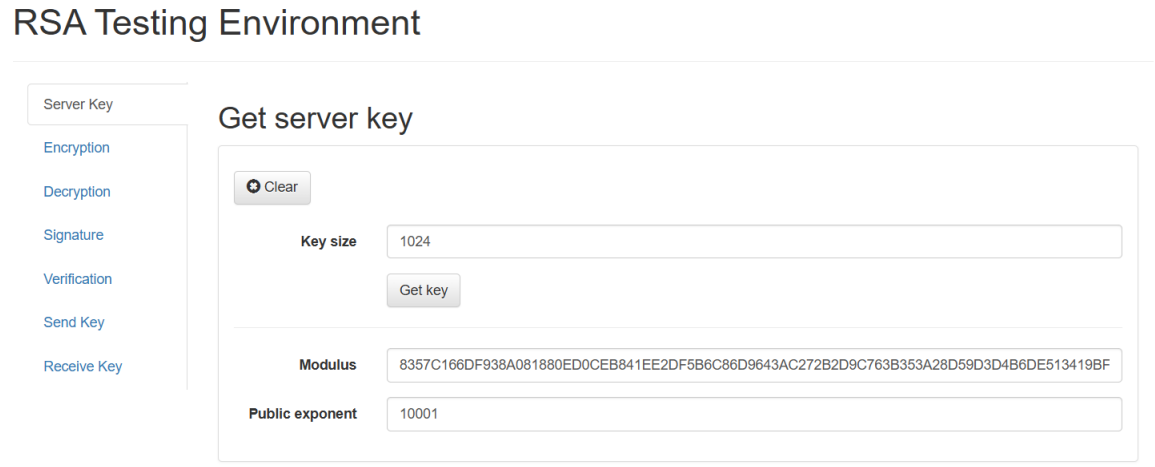


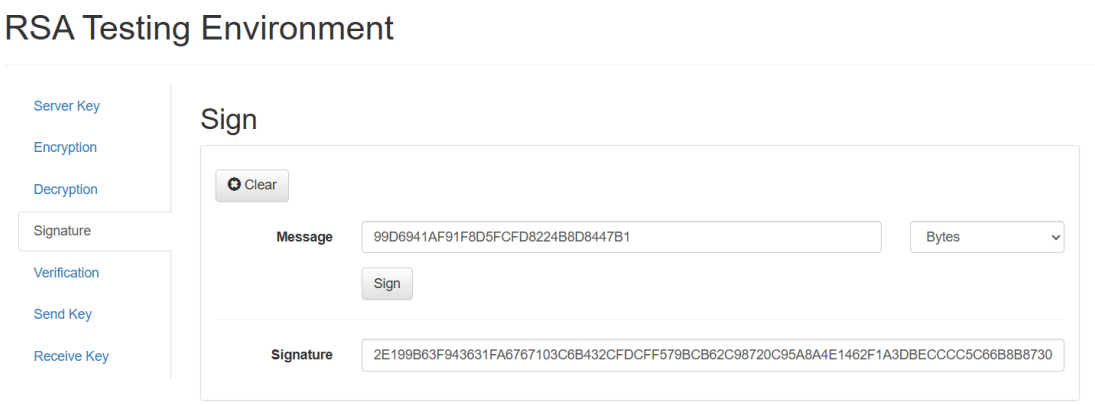
Далі було перевірено зворотний напрямок. На вкладці Encryption тестового середовища (режим Bytes) задано відкритий ключ абонента A: модуль n та відкриту експоненту e = 10001. Як відкрите повідомлення

програмою було обрано M = AB33B7C95EA074E2A14F836D53ABCCB0 (hex). Після натискання Encrypt сервер зашифрував повідомлення й повернув шифртекст C = 48D76BAB1B7097756AFFBE7BE8025F256C701024CD415A46E8E2184CA AF3B7FD5A8671409643CD23C76DB0D70E6F16F75A59409AA1774342B92 BF5580FF725B1.

Отриманий шифртекст було подано на вхід власної функції Decrypt(), яка розшифрувала його за секретним ключем абонента A. Результат розшифрування AB33B7C95EA074E2A14F836D53ABCCB0 повністю збігся з вихідним повідомленням M, порівняння повернуло True. Отже, власна реалізація розшифрування сумісна з реалізацією шифрування на сервері.







Далі було перевірено цифровий підпис, створений сервером. На вкладці Server Key було згенеровано ключ сервера розміром 1024 біти, сервер повернув модуль n_s та відкриту експоненту e_s = 10001 (hex).

Програмою було обрано випадкове повідомлення M = 99D6941AF91F8D5FCFD8224B8D8447B1 (hex). Його було введено на вкладці Sign тестового середовища (режим Bytes). Після натискання Sign сервер підписав повідомлення своїм секретним ключем і повернув підпис S = 2E199B63F943631FA6767103C6B432CFDCFF579BCB62C98720C95A8A4E1462F1A3DBECCCC5C66B8B8730DE3C8CA868C7660829FD7332BFD62F36682FB2A2BC9D3AE2FBE7DF8C6A774F52C1CB4E9C542BE47FCD26DB55666144197489A9ADD0D4193580DCCE8BB4843E4EB10D382B5E5D013AFF7B54D49061C7971B5E3FD3DE32.

Отримані M та S було подано на вхід власної функції Verify() разом з відкритим ключем сервера (n_s, e_s). Функція повернула True, тобто підпис сервера є справжнім. Для контролю повідомлення було змінено на M + 1 і перевірку виконано з тим самим підписом. У цьому випадку Verify() повернула False, оскільки підпис створено для іншого значення повідомлення. Отже, власна реалізація перевірки підпису сумісна з реалізацією підпису на сервері, а зміна підписаного повідомлення виявляється.

```
=== Перевірка із тестовим середовищем ===
1 - Encrypt (наша) -> Decrypt на сервері
2 - Encrypt на сервері -> Decrypt (наша)
3 - Sign на сервері -> Verify (наша)
4 - Sign (наша) -> Verify на сервері
5 - SendKey (наша) -> ReceiveKey на сервері
6 - SendKey на сервері -> ReceiveKey (наша)
0 - вихід
Оберіть пункт: 4

[4] Sign (наша реалізація) -> Verify на сервері

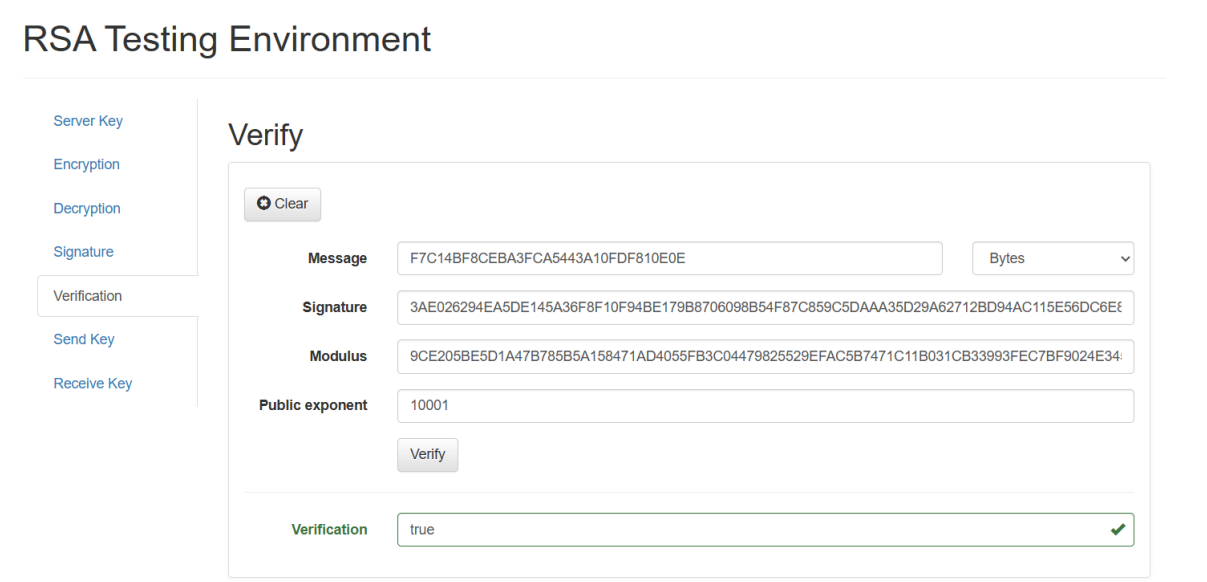
Вкладка Verification (режим Bytes): введіть усі чотири поля
Message M:
F7C14BF8CEBA3FCA5443A10FDF810E0E

Signature S:
3AE026294EA5DE145A36F8F10F94BE179B8706098B54F87C859C5DAAA35D29A62712BD94AC115E56DC6E86F999250D3929B7D8A64A91CE567C0E967AB1290F8E

Modulus (n абонента A):
9CE205BE5D1A47B785B5A158471AD4055FB3C04479825529EFAC5B7471C11B031CB33993FEC7BF9024E3459382B7D16C827105B4AD718F7590FB0F88772DD663

Public exponent (e):
10001

(значення також записано у файл server_values.txt)
Натисніть Verify: сервер має показати true.
```

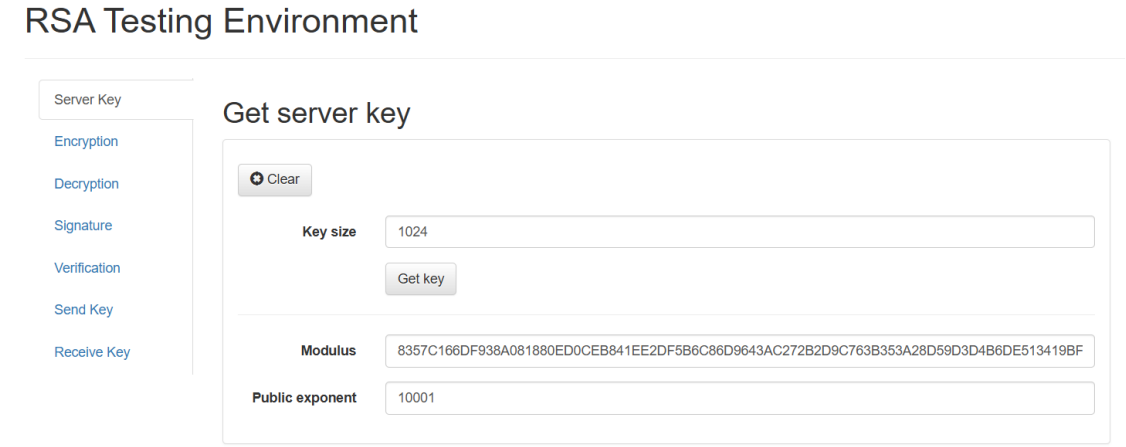


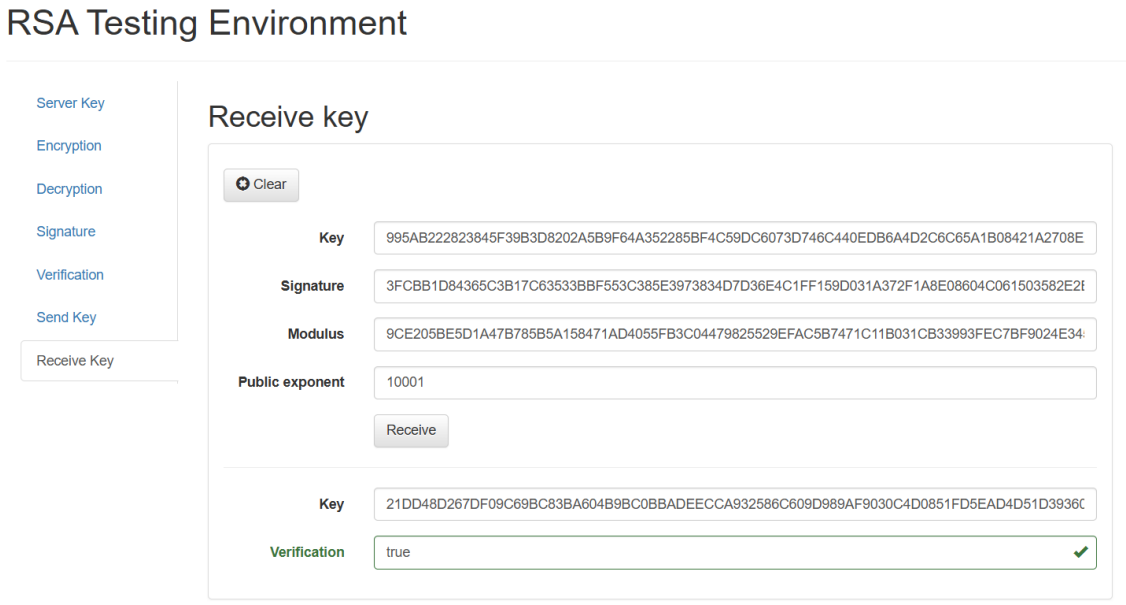
Далі було перевірено цифровий підпис, створений власною реалізацією. Програмою було обрано випадкове повідомлення M = F7C14BF8CEBA3FCA5443A10FDF810E0E (hex) і підписано його власною функцією Sign() за секретним ключем абонента A. Отримано підпис S = 3AE026294EA5DE145A36F8F10F94BE179B8706098B54F87C859C5DAAA35D29A62712BD94AC115E56DC6E86F999250D3929B7D8A64A91CE567C0E967AB1290F8E.

На вкладці Verification тестового середовища (режим Bytes) було введено повідомлення M, підпис S, а також відкритий ключ абонента A: модуль n та відкриту експоненту e = 10001. Після натискання Verify сервер повернув результат true, тобто підтвердив справжність підпису. Отже,

підпис, створений власною реалізацією, коректно перевіряється сервером, і власна реалізація підпису сумісна з реалізацією перевірки на сервері.





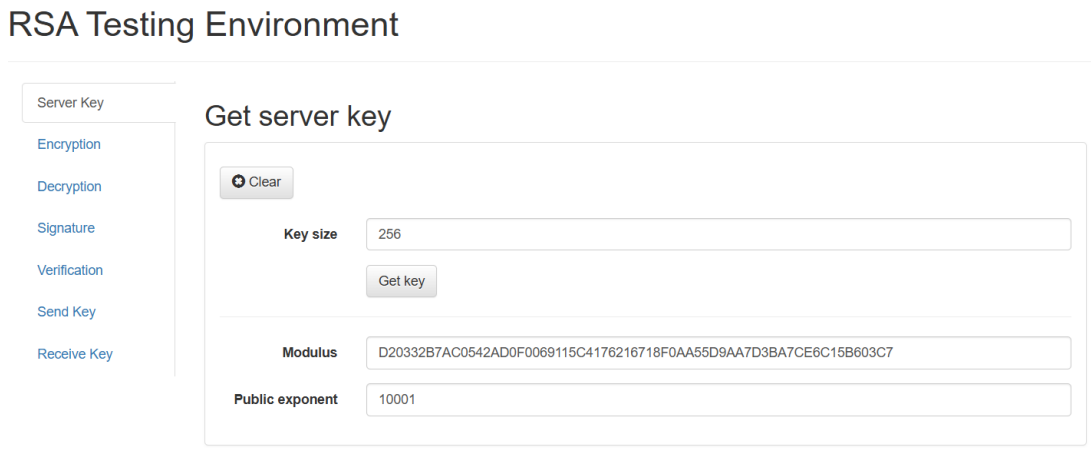


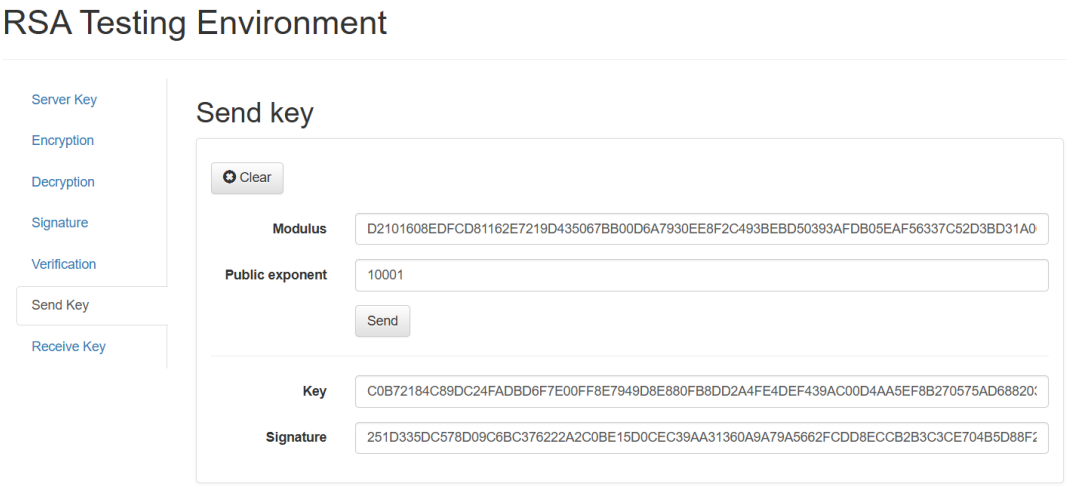
Далі було перевірено протокол розсилання ключів, у якому відправником

виступає власна реалізація (абонент A), а отримувачем сервер. Оскільки модуль відправника не має перевищувати модуль отримувача, на вкладці Server Key було згенеровано ключ сервера розміром 1024 біти, що більше за 512-бітний модуль абонента A. Сервер повернув модуль n_s та відкриту експоненту e_s = 10001 (hex).

Програмою було обрано випадковий ключ 0 < k < n: k = 21DD48D267DF09C69BC83BA604B9BC0BBADEECCA932586C609D989AF9030C4D0851FD5EAD4D51D39360C64263240139D034BD831DAAA71B26D5815F98BC561DB (hex). Власною функцією SendKey() за секретним ключем абонента A та відкритим ключем сервера (n_s, e_s) було сформовано пару (k1, S1), де k1 є зашифрованим ключем, а S1 є зашифрованим підписом.

На вкладці Receive Key тестового середовища було введено Key (k1), Signature (S1), а також відкритий ключ абонента A: модуль n та відкриту експоненту e = 10001. Після натискання Receive сервер розшифрував k1 та S1 своїм секретним ключем і підтвердив підпис (Verification: true). Отриманий сервером ключ збігається з надісланим k (сервер виводить його без урахування початкових нулів). Отже, пара (k1, S1), сформована власною реалізацією SendKey(), коректно приймається сервером.






```
=== Перевірка із тестовим середовищем ===
1 - Encrypt (наша) -> Decrypt на сервері
2 - Encrypt на сервері -> Decrypt (наша)
3 - Sign на сервері -> Verify (наша)
4 - Sign (наша) -> Verify на сервері
5 - SendKey (наша) -> ReceiveKey на сервері
6 - SendKey на сервері -> ReceiveKey (наша)
0 - вихід
Оберіть пункт: 6

[6] SendKey на сервері -> ReceiveKey (наша реалізація, абонент B)
  Спершу Get key на вкладці Server Key з розміром 256 біт (модуль сервера має бути меншим за n1).
  Modulus сервера (вкладка Server Key), hex: D20332B7AC0542AD0F0069115C4176216718F0AA55D9AA7D3BA7CE6C15B603C7
  Public exponent сервера, hex: 10001

Вкладка Send Key: введіть Modulus та Public exponent
Modulus (n1 абонента B):
D2101608EDFCD81162E7219D435067BB00D6A7930EE8F2C493BEBD50393AFDB05EAF56337C52D3BD31A068D0D8498BFF09E09FD86500D43970FEF4B59402C7AB

Public exponent (e1):
10001

(значення також записано у файл server_values.txt)
Натисніть Send, скопіюйте Key та Signature із сайту.
  Key із сайту, hex: C0B72184C89DC24FADBD6F7E00FF8E7949D8E880FB8DD2A4FE4DEF439AC00D4AA5EF8B270575AD68820319C691BFFC0BF814A014E98D08D764776FE2E19A35D6
  Signature із сайту, hex: 251D335DC578D09C6BC376222A2C0BE15D0CEC39AA31360A9A79A5662FCDD8ECCB2B3C3CE704B5D88F26DE71102B474B3D6E420459C1C23806BA41AFD5C2FB64

Ключ прийнято, підпис вірний. k = 832B0046797FBA14
Для зміненого на одиницю Signature результат: None
```

Далі було перевірено протокол розсилання ключів у зворотному напрямку: відправником виступає сервер, а отримувачем власна реалізація (абонент B). Оскільки модуль відправника не має перевищувати модуль отримувача, на вкладці Server Key було згенеровано новий ключ сервера

розміром 256 біт, що менше за 512-бітний модуль n1 абонента B. Сервер повернув модуль n_s = D20332B7AC0542AD0F0069115C4176216718F0AA55D9AA7D3BA7CE6C15B603C7 та відкриту експоненту e_s = 10001 (hex).

На вкладці Send Key тестового середовища було введено відкритий ключ абонента B: модуль n1 та відкриту експоненту e1 = 10001. Після натискання Send сервер сформував випадковий ключ, підписав його своїм секретним ключем, зашифрував ключ і підпис за відкритим ключем абонента B та повернув пару значень Key = C0B72184C89DC24FADBD6F7E00FF8E7949D8E880FB8DD2A4FE4DEF439AC00D4AA5EF8B270575AD68820319C691BFFC0BF814A014E98D08D764776FE2E19A35D6 і Signature = 251D335DC578D09C6BC376222A2C0BE15D0CEC39AA31360A9A79A5662FCDD8ECCB2B3C3CE704B5D88F26DE71102B474B3D6E420459C1C23806BA41AFD5C2FB64.

Отримані значення було подано на вхід власної функції ReceiveKey() разом із секретним ключем абонента B та відкритим ключем сервера (n_s, e_s). Функція розшифрувала k1 та S1 за ключем d1, перевірила підпис за відкритим ключем сервера, ключ було прийнято, підпис виявився вірним, отримано ключ k = 832B0046797FBA14 (hex). Для контролю значення Signature було змінено на одиницю: у цьому випадку функція ReceiveKey() повернула None, тобто ключ було відхилено. Отже, власна реалізація отримання ключа сумісна з реалізацією відправлення ключа на сервері, а спотворення підпису виявляється.

Контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них?

Спочатку кандидат формується випадково за допомогою random.getrandbits() з виставленим старшим бітом (це гарантує задану довжину 256 біт) та молодшим бітом (це гарантує непарність). Далі виконуються пробні ділення на малі прості числа від 2 до 37: якщо кандидат ділиться на одне з них і не дорівнює йому, він відкидається без виконання складніших обчислень. Після цього застосовується тест Міллера–Рабіна: число n − 1 подається у вигляді d·2^s з непарним d, а для

кожної з k = 40 випадкових основ x перевіряється, що gcd(x, n) = 1 (інакше число складене) і що виконується умова сильної псевдопростоти: x^d ≡ ±1 (mod n) або x^(d·2^r) ≡ −1 (mod n) для деякого r < s. Якщо жодна з умов не виконується, число складене. Тест односторонній: для простого числа він завжди повертає «просте», а складене число проходить одну основу з імовірністю не більше 1/4, тому після 40 раундів імовірність помилки не перевищує 4^(−40). Кандидати, що не пройшли перевірку, відкидаються, і генерація повторюється. Окремо перевіряється, що числа p і q у парі різні.

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації?

Для коректної роботи RSA мають виконуватися такі умови: p і q є різними простими числами; n = p·q; φ(n) = (p − 1)(q − 1); відкрита експонента задовольняє 2 ≤ e ≤ φ(n) − 1 та gcd(e, φ(n)) = 1; секретна експонента є оберненою до e за модулем φ(n), тобто e·d ≡ 1 (mod φ(n)). Додатково для протоколу розсилання ключів потрібно, щоб n ≤ n1.

У реалізації простота p, q, p1, q1 була перевірена тестом Міллера–Рабіна (результат True для всіх чотирьох чисел). Функція GenerateKeyPair() відхиляє випадок p = q, використовує e = 65537 і, якщо gcd(e, φ(n)) ≠ 1, обирає іншу допустиму експоненту. Секретна експонента d обчислюється функцією modular_inverse(), яка повідомляє про помилку, якщо оберненого елемента не існує. Програма вивела перевірки n = p·q, φ(n) = (p − 1)(q − 1), 2 ≤ e ≤ φ(n) − 1, gcd(e, φ(n)) = 1 та e·d mod φ(n) = 1, а також умову n ≤ n1 (результат True).

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA?

Для Encrypt() потрібні повідомлення M та відкритий ключ адресата (n, e). Для Decrypt() потрібні криптограма C та секретний ключ власника (d, p, q), з яких використовується d та модуль n = p·q. Для Sign() потрібні повідомлення M та секретний ключ відправника. Для Verify() потрібні повідомлення M, підпис S та відкритий ключ відправника (n, e).

Кожна з цих операцій є піднесенням до степеня за модулем n, тому їй потрібні лише відповідна експонента і модуль. Операціям з відкритими

ключами секретні дані не потрібні взагалі: інакше їх можна було б виконати лише власнику ключа, що суперечить ідеї асиметричної криптографії. Операціям з секретним ключем не потрібне φ(n): воно, як і числа p та q, потрібне лише для побудови ключів, а його розкриття дозволило б обчислити секретну експоненту й скомпрометувати систему. Тому кожна процедура отримує лише необхідний мінімум даних.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника?

Конфіденційність забезпечується шифруванням за відкритим ключем отримувача B (e1, n1): ключ k1 = k^e1 mod n1 і підпис S1 = S^e1 mod n1 може розшифрувати лише власник секретної експоненти d1, тобто абонент B. Оскільки підпис також передається в зашифрованому вигляді, його не бачить і сторонній спостерігач каналу.

Підтвердження справжності відправника забезпечується цифровим підписом S = k^d mod n, який може створити лише власник секретного ключа d, тобто абонент A. Отримувач B перевіряє його за відкритим ключем відправника (e, n), обчислюючи S^e mod n і порівнюючи результат з k. Умова n ≤ n1 гарантує, що підпис S < n коректно зашифрується за модулем n1.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім?

Одержувач B спочатку розшифровує отримані k1 та S1 своїм секретним ключем d1 (при цьому перевіряється, що 0 ≤ k1, S1 < n1), а потім за відкритим ключем відправника перевіряє, що S^e mod n = k. Лише якщо ця рівність виконується, ключ k приймається. Якщо перевірка не пройдена, функція ReceiveKey() повертає None, тобто ключ відхиляється. Це було підтверджено експериментально: при зміні S1 на одиницю та при перевірці за чужим відкритим ключем ключ відхилявся.

Слід зазначити, що сам протокол не гарантує, що відкритий ключ відправника справді належить A (для цього потрібні сертифікати або

інший канал довіри), а також не захищає від повторного надсилання перехопленого повідомлення.

Висновок

У ході виконання практикуму було вивчено криптосистему RSA та електронний підпис, а також методи генерації параметрів для асиметричних криптосистем. Самостійно реалізовано тест Міллера–Рабіна з пробними діленнями та генератор 256-бітних простих чисел, за якими побудовано ключові пари абонентів A і B з 512-бітними модулями та e = 65537. Реалізовано шифрування, розшифрування, створення й перевірку цифрового підпису: розшифрування повертає вихідне повідомлення, перевірка підтверджує справжність підпису, а зміна повідомлення виявляється. На їх основі реалізовано протокол конфіденційного розсилання ключів із підтвердженням справжності: конфіденційність забезпечується шифруванням за відкритим ключем отримувача, а справжність відправника підписом за його секретним ключем. Роботу реалізації перевірено із тестовим середовищем в обох напрямках, усі шість перевірок успішні. Реалізовано «підручникове» RSA без доповнення та гешування і з некриптографічним генератором випадкових чисел, тому для практичного захисту інформації воно не придатне.